

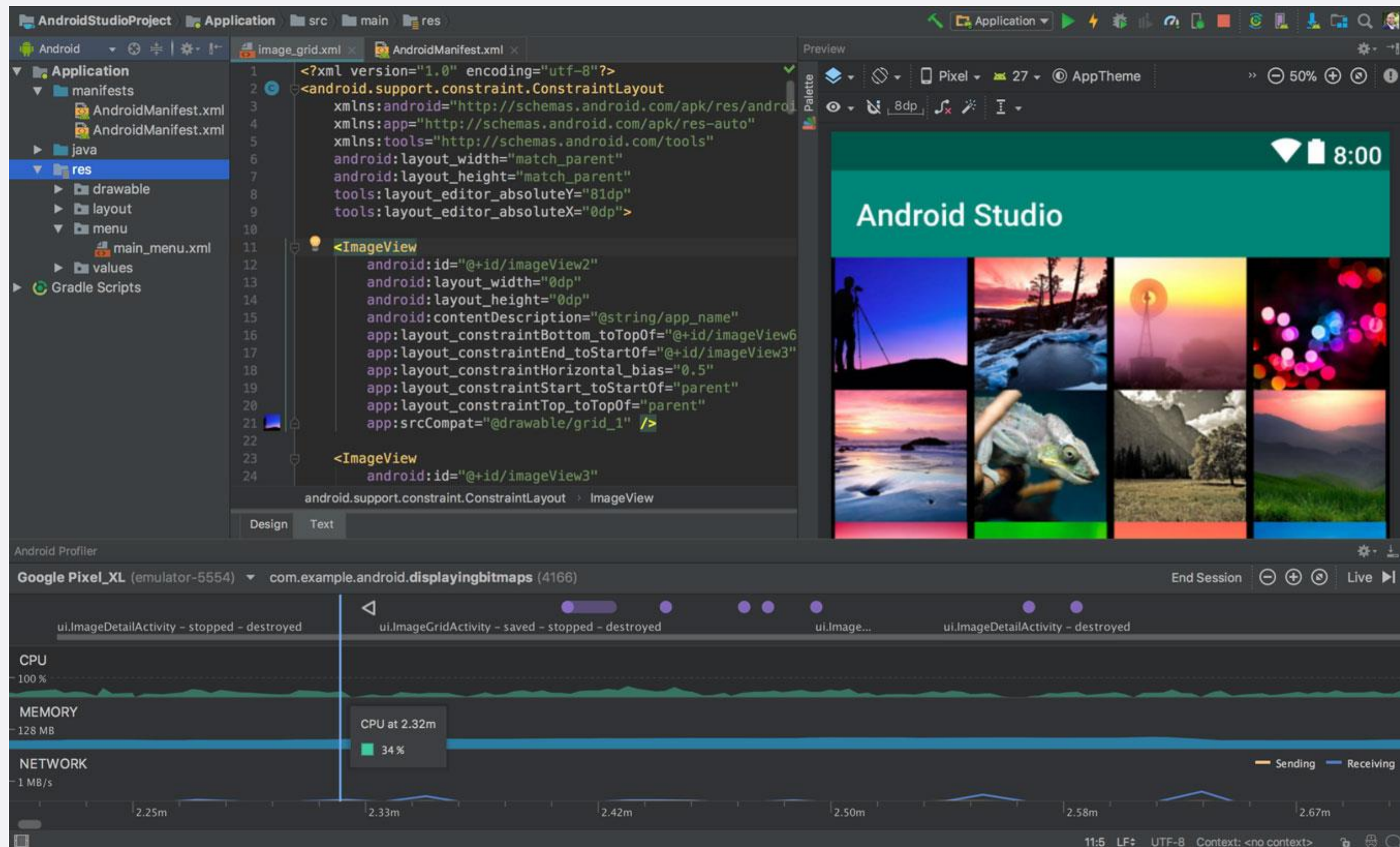
Corso di Laboratorio di Programmazione per Sistemi Mobile e Tablet

Docente: Dr. Mauro Dragoni

Dipartimento di Ingegneria e Scienze dell'Informazione
Anno Accademico 2025/2026

Android studio

<https://developer.android.com/studio>



Android studio

- **dx**
 - converte i file .class di Java in file .dex (Dalvik Executable).
- **aapt** (Android Asset Packaging Tool)
 - crea i pacchetti delle applicazioni Android in formato .apk (Android Package).
- **adb** (Android debug bridge)
 - command-line tool per fare debug direttamente sul device.
- **ADT** (Android Development Tools for Eclipse)
 - Uno strumento di sviluppo fornito da Google per eseguire la conversione automatica da file .classe a file .dex e per creare l'apk durante la distribuzione. Fornisce inoltre strumenti di debug e un emulatore di dispositivo Android.

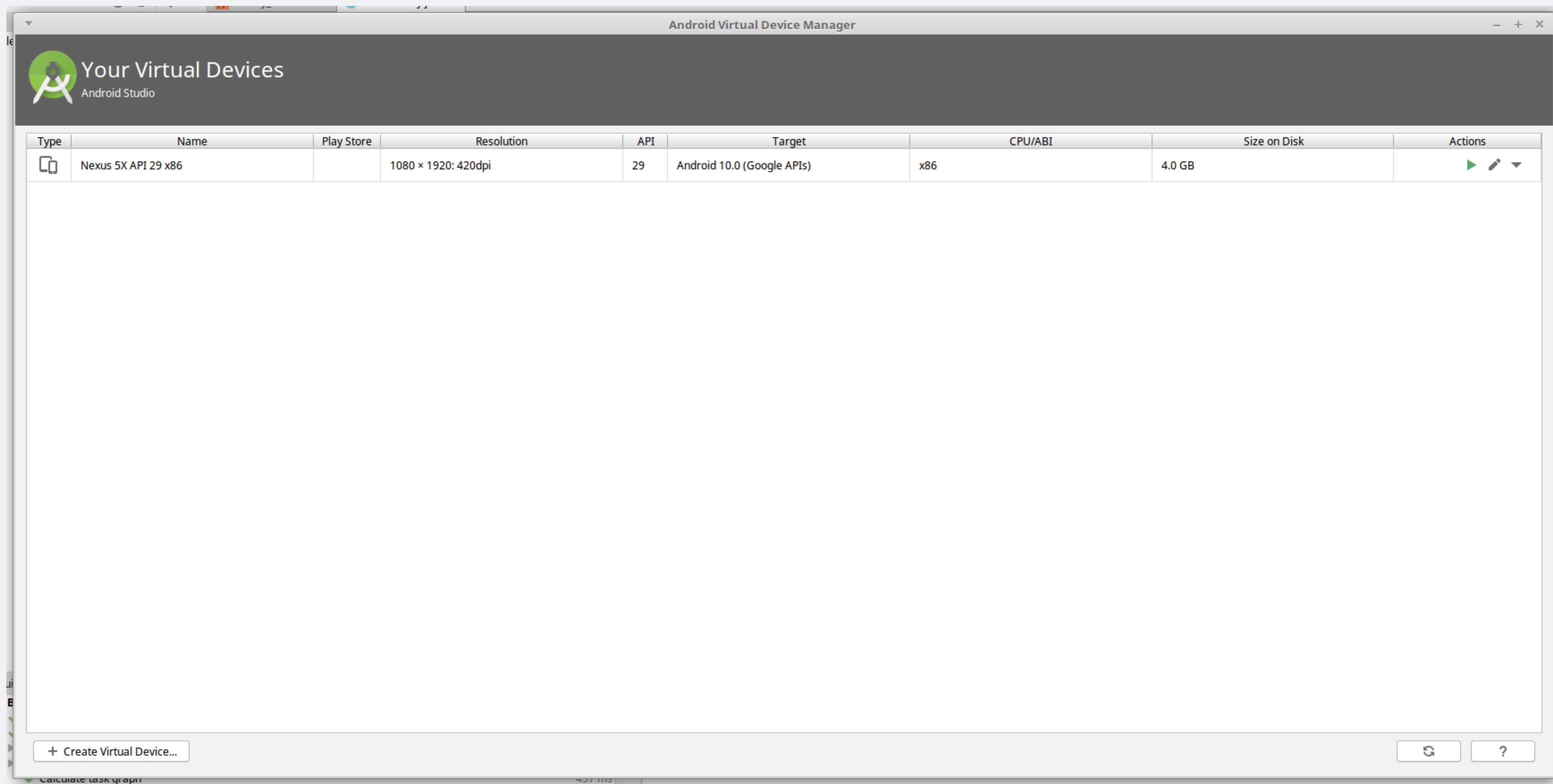
Android Virtual Device

- Un emulatore che ti consente di simulare un dispositivo reale partendo dalla definizione delle caratteristiche hardware e software.
- Un AVD e' costituito da:
 - Un profilo hardware: Definisce le funzionalità hardware del dispositivo virtuale (se ha un fotocamera, una tastiera QWERTY fisica o un tastierino numerico, quanto memoria ha ecc.
 - Una mappatura su un'immagine di sistema: e' possibile definire quale versione della piattaforma Android verra' eseguita su dispositivo virtuale
 - Altre opzioni: la skin dell'emulatore (dimensioni dello schermo, aspetto, ecc.), scheda SD emulata
 - Un'area di archiviazione dedicata sulla macchina di sviluppo: i dati utente del dispositivo (applicazioni installate, impostazioni e cosi' via) e le schede SD emulate sono memorizzate in quest'area.

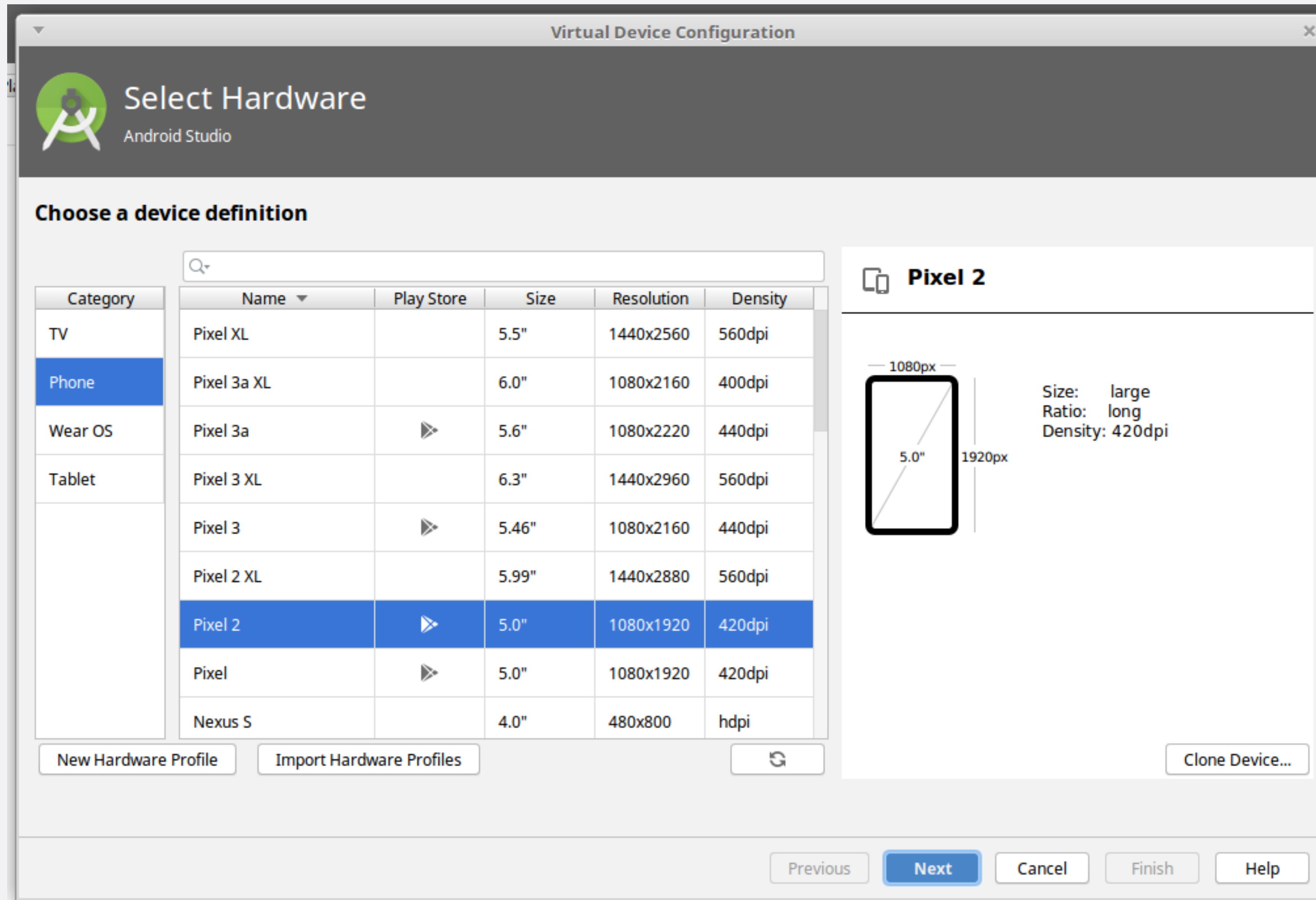
Android Virtual Device

- Possono essere create:
 - dalla linea di comando (comando «avdmanager» nella cartella «android_sdk/cmdline-tools/version/bin/avdmanager»).
 - tramite l'utilizzo di Android studio.

Android Virtual Device



Android Virtual Device



Android Virtual Device

Virtual Device Configuration

System Image
Android Studio

Select a system image

Recommended

x86 Images

Other Images

Release Name	API Level	ABI	Target
R Download	R	x86	Android API R (Google APIs)
Q	29	x86	Android 10.0 (Google APIs)
Pie Download	28	x86	Android 9.0 (Google APIs)
Oreo Download	27	x86	Android 8.1 (Google APIs)
Oreo Download	26	x86	Android 8.0 (Google APIs)
Nougat Download	25	x86	Android 7.1.1 (Google APIs)
Nougat Download	24	x86	Android 7.0 (Google APIs)
Marshmallow Download	23	x86	Android 6.0 (Google APIs)
Lollipop Download	22	x86	Android 5.1 (Google APIs)

Q



API Level

29

Android

10.0

Google Inc.

System Image

x86

We recommend these images because they run the fastest and support Google APIs.

Questions on API level?
[See the API level distribution chart](#)

Previous

Next

Cancel

Finish

Help

Android Virtual Device

Virtual Device Configuration



Android Virtual Device (AVD)
Android Studio

Verify Configuration

AVD Name

Nexus 6P API 29

 Nexus 6P

5.7 1440x2560 560dpi

Change...

 Q

Android 10.0 x86

Change...

Startup orientation

 Portrait

 Landscape

Emulated Performance

Graphics: Automatic

Device Frame

☒ Enable Device Frame

Show Advanced Settings

AVD Name

The name of this AVD.

Previous

Next

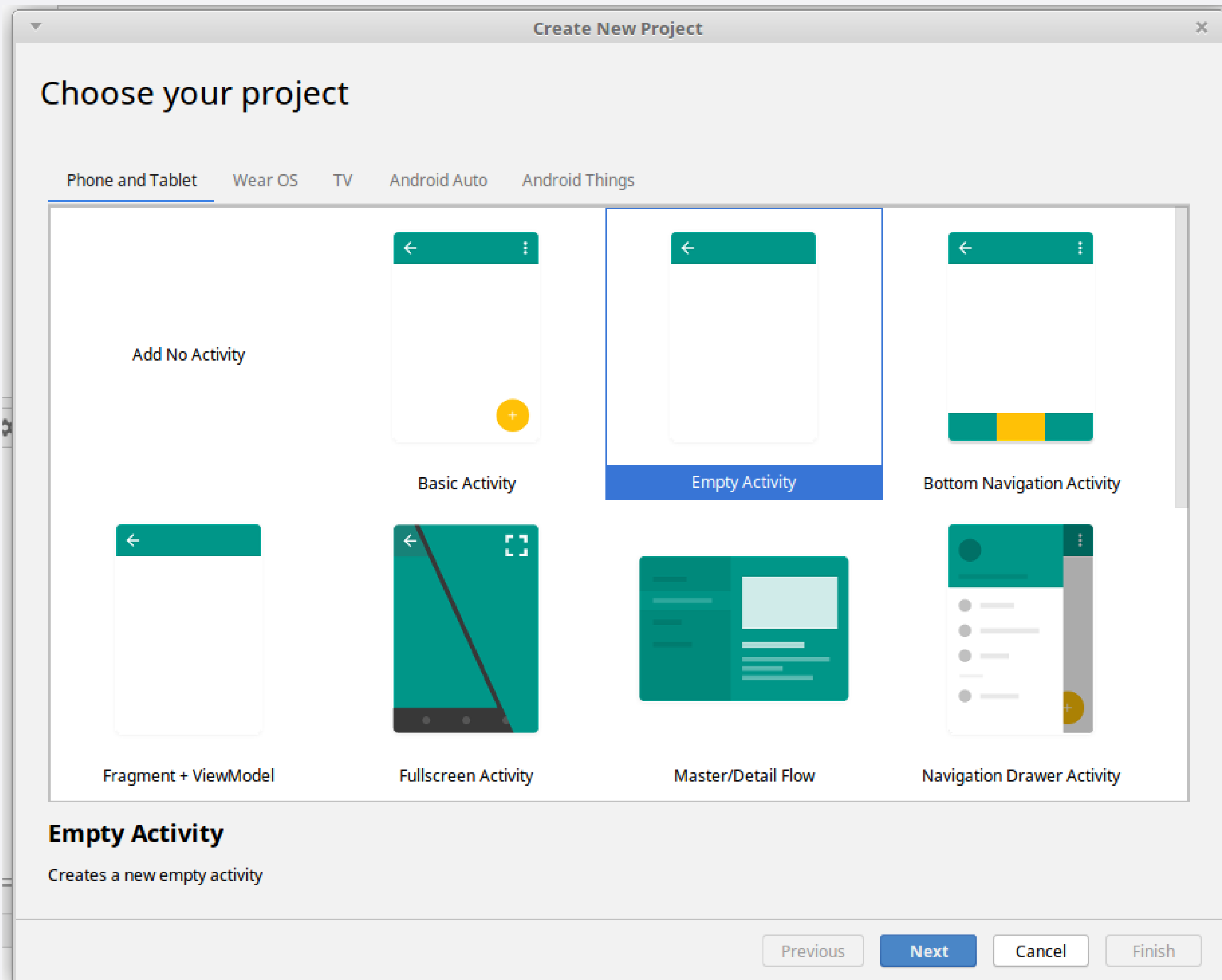
Cancel

Finish

Help

page
09

hello world Android!



hello world Android!

New Project

Empty Activity

Create a new empty activity with Jetpack Compose

Name

Hello World

Package name

com.example.helloworld

Save location

/home/drago/Documents/java_projects/android

Minimum SDK

API 24 ("Nougat"; Android 7.0)

Your app will run on approximately 99.2% of devices.

Help me choose

Build configuration language ?

Kotlin DSL (build.gradle.kts) [Recommended]

Previous

Next

Cancel

Finish

instant app?

- Permettono l'accesso a determinate funzionalità delle vostre applicazioni senza che l'utente la debba installare.
 - Le funzionalità esposte sono associate ad un URL che, una volta richiamato (ad esempio da Web) permette di visualizzare/utilizzare la funzionalità stessa.
- Vantaggi
 - rimozione delle barriere di accesso alla vostra applicazione;
 - migliora il raggiungimento di nuovi utenti;
 - aumenta l'attrattiva della vostra applicazione qualora implementiate funzionalità legate al contesto spazio-temporale di utilizzo dell'applicazione.

instant app?



Avviso: Google Play Instant non sarà più disponibile. A partire da dicembre 2025, le app istantanee non potranno essere pubblicate tramite Google Play e tutte le API istantanee di Google Play Services non funzioneranno più. Gli utenti non riceveranno più app istantanee da Play utilizzando alcun meccanismo.

Stiamo apportando questa modifica in base al feedback degli sviluppatori e ai nostri continui investimenti per migliorare l'ecosistema dall'introduzione di Google Play Instant.

Per continuare a ottimizzare per la crescita degli utenti, invitiamo gli sviluppatori a indirizzare gli utenti alla loro app o al loro gioco normale utilizzando i deep link per reindirizzarli a percorsi o funzionalità specifici, se pertinenti.

hello world Android!

```
activity_main.xml x MainActivity.java x
1 package com.example.test03;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import android.os.Bundle;
5 import android.widget.TextView;
6
7 public class MainActivity extends AppCompatActivity {
8
9     @Override
10    protected void onCreate(Bundle savedInstanceState) {
11        super.onCreate(savedInstanceState);
12        TextView tv = new TextView(context: this);
13        tv.setText("Hello, Android");
14        setContentView(tv);
15    }
16 }
17
```

```
activity_main.xml x MainActivity.java x
1 package com.example.test02;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import android.os.Bundle;
5
6 public class MainActivity extends AppCompatActivity {
7
8     @Override
9    protected void onCreate(Bundle savedInstanceState) {
10        super.onCreate(savedInstanceState);
11        setContentView(R.layout.activity_main);
12    }
13 }
14
```

hello world Android!

```
activity_main.xml x MainActivity.java x
1 package com.example.test03;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import android.os.Bundle;
5 import android.widget.TextView;
6
7 public class MainActivity extends AppCompatActivity {
8
9     @Override
10    protected void onCreate(Bundle savedInstanceState) {
11        super.onCreate(savedInstanceState);
12        TextView tv = new TextView(context: this);
13        tv.setText("Hello, Android");
14        setContentView(tv);
15    }
16 }
17
```

```
activity_main.xml x MainActivity.java x
1 package com.example.test02;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import android.os.Bundle;
5
6 public class MainActivity extends AppCompatActivity {
7
8     @Override
9    protected void onCreate(Bundle savedInstanceState) {
10        super.onCreate(savedInstanceState);
11        setContentView(R.layout.activity_main);
12    }
13 }
14
```

la classe Activity

- **Activity**: classe base per la creazione di applicazioni mobile. Non e' raccomandato ereditare direttamente da questa classe in quanto le sue specializzazioni sono nettamente migliori.
- **ActionBarActivity**: ci fu un momento in cui rimpiazzo' la classe Activity in quanto permetteva di manipolare piu' facilmente le ActionBar all'interno delle applicazioni.
- **AppCompatActivity**: e' la best practice consigliata per la programmazione classica basata sull'uso dei layout XML in quanto l'utilizzo della classe ActionBar non e' incoraggiata a prescindere. Si preferisce l'utilizzo della classe Toolbar (visto che e' proprio il rimpiazzo della classe ActionBar). AppCompatActivity eredita direttamente da FragmentActivity, quindi, qualora necessitate di utilizzare i Fragments e' possibile farlo (tramite il Fragment Manager). AppCompatActivity e' valida per tutte le versioni delle API, non solo dalla 16 e successive.
- **ComponentActivity**: e' la best practice consigliata per la programmazione basata sull'uso del Jetpack Compose. Risiede nella libreria androidx.activity e fornisce l'infrastruttura di base su cui si fonda lo sviluppo Android moderno: gestione del ciclo di vita, supporto per ViewModel, SavedStateRegistry, API per i risultati delle attività, gestione della pressione del tasto Indietro e, soprattutto, setContent per Jetpack Compose. Non si porta dietro alcun retaggio del sistema View legacy oltre a ciò che è essenziale.

Activity

- Corrisponde a ciascuna schermata dell'interfaccia utente
- Ogni volta che si usa un'applicazione generalmente si interagisce con una o più "pagine" mediante le quali si consultano dati o si immettono input.
- La realizzazione di Activity è il punto di partenza di ogni applicazione Android visto che è il componente con cui l'utente ha il contatto più diretto.
- Le Activity nel loro complesso costituiscono il flusso in cui l'utente si inoltra per sfruttare le funzionalità messe a disposizione.
- Questo richiede pertanto non solo la realizzazione delle interfacce in sé, ma anche una corretta progettazione della navigazione tra di esse offrendo la possibilità di sfogliarne i contenuti e risalirli gerarchicamente in maniera coerente.
- A differenza dei sistemi operativi per PC, non è prevista la possibilità di avere più finestre aperte contemporaneamente (problema parzialmente risolto con i Fragment)
 - Al passaggio da una Activity ad un'altra, l'Activity esistente viene messa in pausa

hello world Android!

```
activity_main.xml x MainActivity.java x
1 package com.example.test03;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import android.os.Bundle;
5 import android.widget.TextView;
6
7 public class MainActivity extends AppCompatActivity {
8
9     @Override
10    protected void onCreate(Bundle savedInstanceState) {
11        super.onCreate(savedInstanceState);
12        TextView tv = new TextView(context: this);
13        tv.setText("Hello, Android");
14        setContentView(tv);
15    }
16 }
17
```

```
activity_main.xml x MainActivity.java x
1 package com.example.test02;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import android.os.Bundle;
5
6 public class MainActivity extends AppCompatActivity {
7
8     @Override
9    protected void onCreate(Bundle savedInstanceState) {
10        super.onCreate(savedInstanceState);
11        setContentView(R.layout.activity_main);
12    }
13 }
14
```

Activity

onCreate(Bundle b)

- Richiamata richiamata all'avvio della Activity.
- Rappresenta il punto dove dovrebbe andare la maggior parte delle inizializzazioni.
- Se la Activity viene re-inizializzata dopo essere stata chiusa in precedenza, `Bundle`, contiene i dati forniti dall'ultima chiamata del metodo `onSaveInstanceState(Bundle)`, altrimenti `e' null`.
- Nota: un `Bundle` e' una sorta di contenitore di dati serializzati.

hello world Android!

```
activity_main.xml x MainActivity.java x
1 package com.example.test03;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import android.os.Bundle;
5 import android.widget.TextView;
6
7 public class MainActivity extends AppCompatActivity {
8
9     @Override
10    protected void onCreate(Bundle savedInstanceState) {
11        super.onCreate(savedInstanceState);
12        TextView tv = new TextView(context: this);
13        tv.setText("Hello, Android");
14        setContentView(tv);
15    }
16 }
17
```

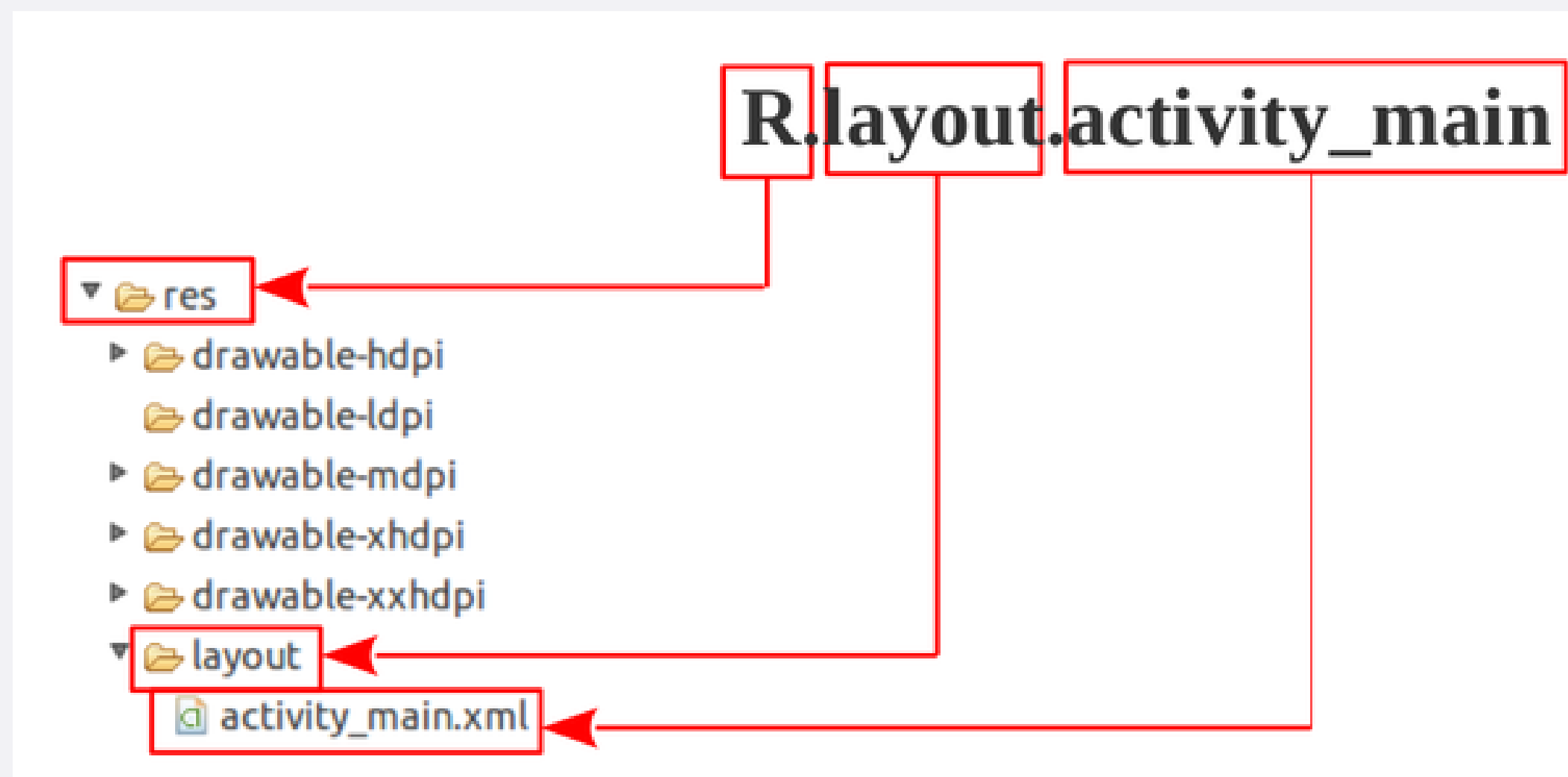
```
activity_main.xml x MainActivity.java x
1 package com.example.test02;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import android.os.Bundle;
5
6 public class MainActivity extends AppCompatActivity {
7
8     @Override
9    protected void onCreate(Bundle savedInstanceState) {
10        super.onCreate(savedInstanceState);
11        setContentView(R.layout.activity_main);
12    }
13 }
14
```


- Regola importante: bisognerebbe sempre externalizzare le risorse (ad es. immagini ed etichette degli elementi grafici) dal codice dell'applicazione, in modo da:
 - mantenerli in modo indipendente.
 - fornire risorse alternative, ad esempio:
 - lingue differenti
 - dimensioni dello schermo diverse
- Le risorse devono essere organizzate nella cartella «**res/**» con varie sottodirectory che raggruppano risorse per tipo e configurazione.

- tra le risorse piu' comuni troviamo:
 - **layout**: contiene l'architettura grafica dei componenti dell'interfaccia utente. Sono definiti in XML in una maniera simile a come viene usato HTML per strutturare le pagine web.
 - **values**: contiene stringhe, colori, dimensioni e altre tipologie di valori che potranno essere usate in ulteriori risorse o nel codice Java. Importante notare che questi valori costituiranno il contenuto di appositi tag XML (<string>, <dimen>, etc.) raggruppati in files dal nome solitamente indicativo: *strings.xml*, *dimens.xml*, *colors.xml* e via dicendo. Tali nomi sono frutto di pura convenzione ma il programmatore puo' scegliere liberamente come chiamarli.
 - **drawable**: sono immagini nei formati piu' comuni o, cosa che puo' stupire, configurate in XML.

classe R.java

- Quando viene compilata l'applicazione, aapt genera il file `R.java` che contiene gli ID risorsa per tutto il contenuto nella directory «**res/**».
- Per ogni tipo di risorsa, esiste una sottoclasse R (per esempio, `R.layout` per tutte le risorse di layout) e per ogni risorsa di quel tipo, esiste un numero intero statico (per esempio, `R.layout.activity_main`).
- Questo numero intero e' l'identificativo della risorsa che e' possibile utilizzare per recuperare la risorsa.



classe R.java



```
activity_main.xml x MainActivity.java x R.java x
Files under the "build" folder are generated and should not be edited.
1  /* AUTO-GENERATED FILE. DO NOT MODIFY.
2
3  * This class was automatically generated by the
4  * gradle plugin from the resource data it found. It
5  * should not be modified by hand.
6  */
7  package androidx.loader;
8
9  public final class R {
10     private R() {}
11
12     public static final class attr {
13         private attr() {}
14
15         public static final int alpha = 0x7f020027;
16         public static final int font = 0x7f02007a;
17         public static final int fontProviderAuthority = 0x7f02007c;
18         public static final int fontProviderCerts = 0x7f02007d;
19         public static final int fontProviderFetchStrategy = 0x7f02007e;
20         public static final int fontProviderFetchTimeout = 0x7f02007f;
21         public static final int fontProviderPackage = 0x7f020080;
22         public static final int fontProviderQuery = 0x7f020081;
23         public static final int fontStyle = 0x7f020082;
24         public static final int fontVariationSettings = 0x7f020083;
25         public static final int fontWeight = 0x7f020084;
26         public static final int ttcIndex = 0x7f02013c;
27     }
28     public static final class color {
29         private color() {}
30
31         public static final int notification_action_color_filter = 0x7f04003f;
32         public static final int notification_icon_bg_color = 0x7f040040;
33         public static final int ripple_material_light = 0x7f04004a;
34         public static final int secondary_text_default_material_light = 0x7f04004c;
35     }
36     public static final class dimen {
37         private dimen() {}
38
```


accesso risorse

- nei nomi delle cartelle delle risorse si possono utilizzare diversi suffissi per raggruppare le varie risorse in base alla tipologia di schermo, lingua del dispositivo, ecc.

<https://developer.android.com/guide/topics/resources/providing-resources>

- le risorse sono accessibili sia da codice Java che da altre risorse definite in XML:
 - in Java: tramite `R.tipo_risorsa.nome_risorsa`;
 - in XML: `@tipo_risorsa/nome_risorsa`.

Esempio:

```
<resources>
    <string name="app_name">MyApp</string>
</resources>
```

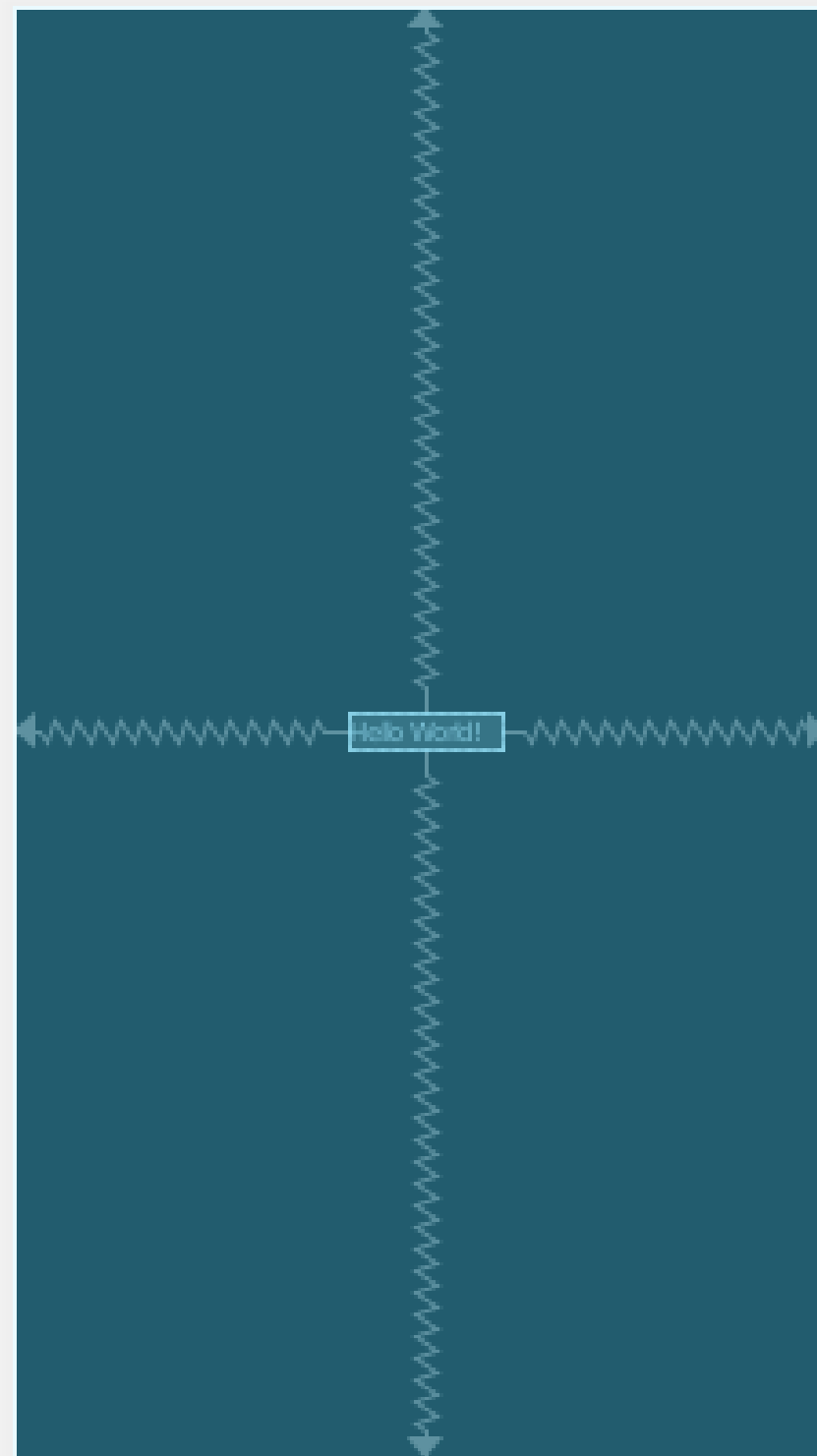
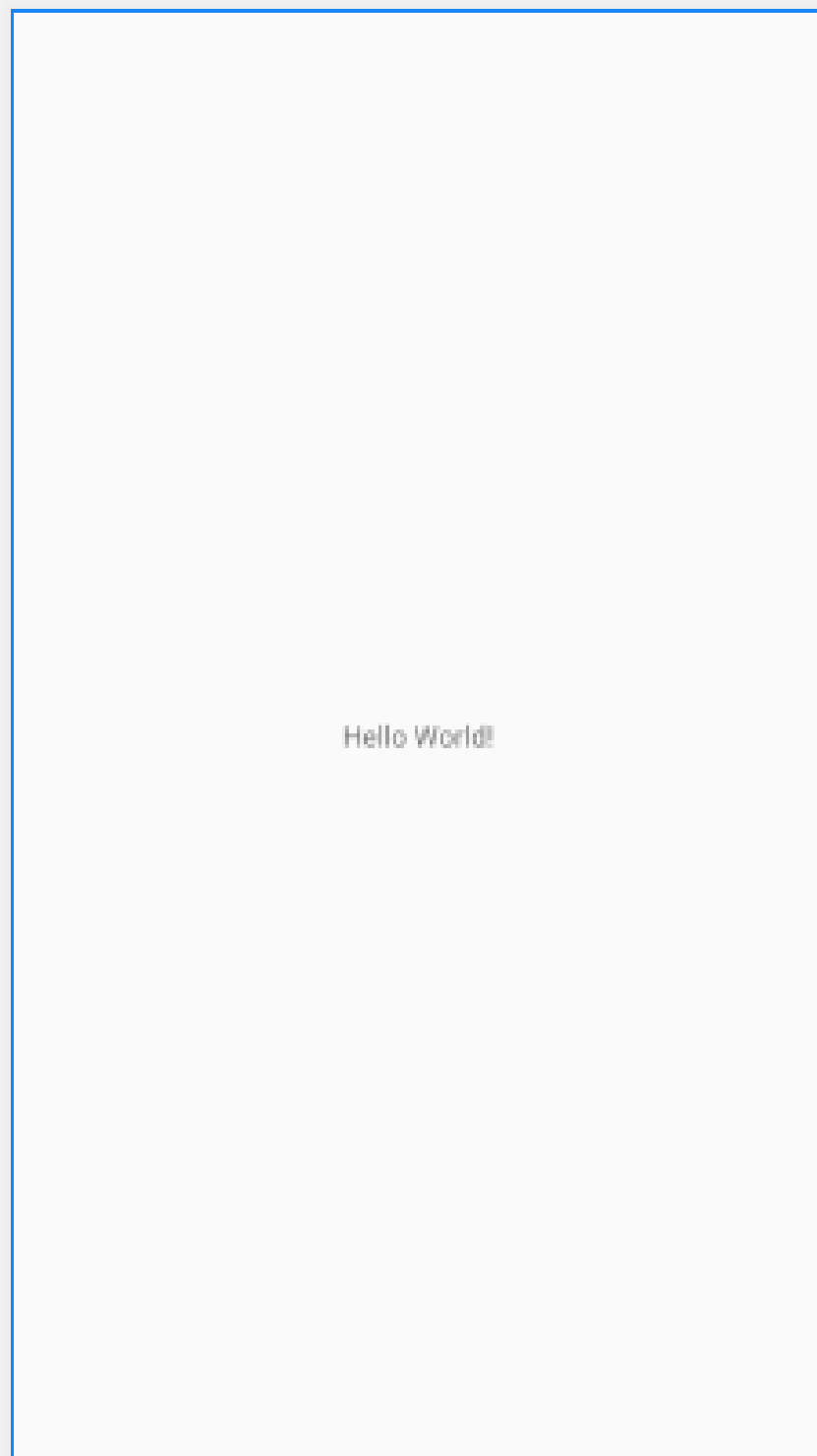
potrà essere recuperata, in Java, mediante `R.string.appname` o dall'interno di altre risorse XML con `@string/appname`.

hello world Android!

```
activity_main.xml x MainActivity.java x
1 package com.example.test03;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import android.os.Bundle;
5 import android.widget.TextView;
6
7 public class MainActivity extends AppCompatActivity {
8
9     @Override
10    protected void onCreate(Bundle savedInstanceState) {
11        super.onCreate(savedInstanceState);
12        TextView tv = new TextView(context: this);
13        tv.setText("Hello, Android");
14        setContentView(tv);
15    }
16 }
17
```

```
activity_main.xml x MainActivity.java x
1 package com.example.test02;
2
3 import androidx.appcompat.app.AppCompatActivity;
4 import android.os.Bundle;
5
6 public class MainActivity extends AppCompatActivity {
7
8     @Override
9    protected void onCreate(Bundle savedInstanceState) {
10        super.onCreate(savedInstanceState);
11        setContentView(R.layout.activity_main);
12    }
13 }
14
```

layout

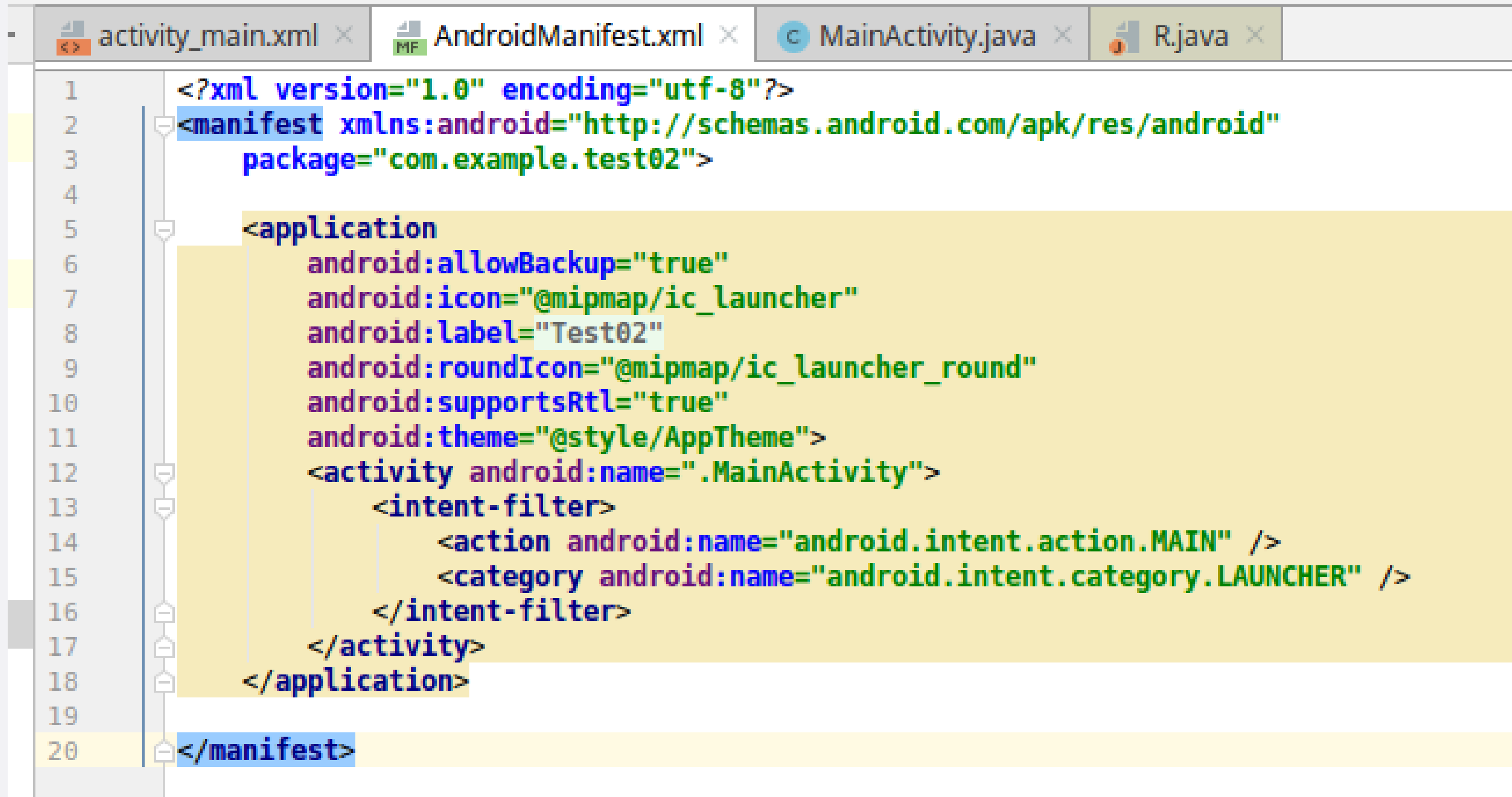


```
activity_main.xml x AndroidManifest.xml x MainActivity.java x R.java x
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
3   xmlns:app="http://schemas.android.com/apk/res-auto"
4   xmlns:tools="http://schemas.android.com/tools"
5   android:layout_width="match_parent"
6   android:layout_height="match_parent"
7   tools:context=".MainActivity">
8
9   <TextView
10     android:layout_width="wrap_content"
11     android:layout_height="wrap_content"
12     android:text="Hello World!"
13     app:layout_constraintBottom_toBottomOf="parent"
14     app:layout_constraintLeft_toLeftOf="parent"
15     app:layout_constraintRight_toRightOf="parent"
16     app:layout_constraintTop_toTopOf="parent" />
17
18 </androidx.constraintlayout.widget.ConstraintLayout>
```

TextView... solo una nota

- Visualizza il testo all'utente e, facoltativamente, gli consente di modificarlo.
- Un TextView e' un editor di testo completo, ma la classe base e' configurata per non consentire l'editing. (vedere EditText per una sottoclasse che configura la vista di testo per la modifica).
- Le varie View le vedremo in seguito...

manifesto dell'applicazione



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     package="com.example.test02">
4
5     <application
6         android:allowBackup="true"
7         android:icon="@mipmap/ic_launcher"
8         android:label="Test02"
9         android:roundIcon="@mipmap/ic_launcher_round"
10        android:supportsRtl="true"
11        android:theme="@style/AppTheme">
12        <activity android:name=".MainActivity">
13            <intent-filter>
14                <action android:name="android.intent.action.MAIN" />
15                <category android:name="android.intent.category.LAUNCHER" />
16            </intent-filter>
17        </activity>
18    </application>
19
20 </manifest>
```

hello world Jetpack Compose edition...

kotlin

```
class MainActivity : ComponentActivity() {
```

kotlin

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)
```

kotlin

```
enableEdgeToEdge()
```

kotlin

```
setContent {  
    HelloWorldTheme {  
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
            Greeting(  
                name = "Android",  
                modifier = Modifier.padding(innerPadding)  
            )  
        }  
    }  
}
```

hello world Jetpack Compose edition...

kotlin

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

kotlin

```
setContent {
    HelloWorldTheme {
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
            Greeting(
                name = "Android",
                modifier = Modifier.padding(innerPadding)
            )
        }
    }
}
```

hello world Jetpack Compose edition...

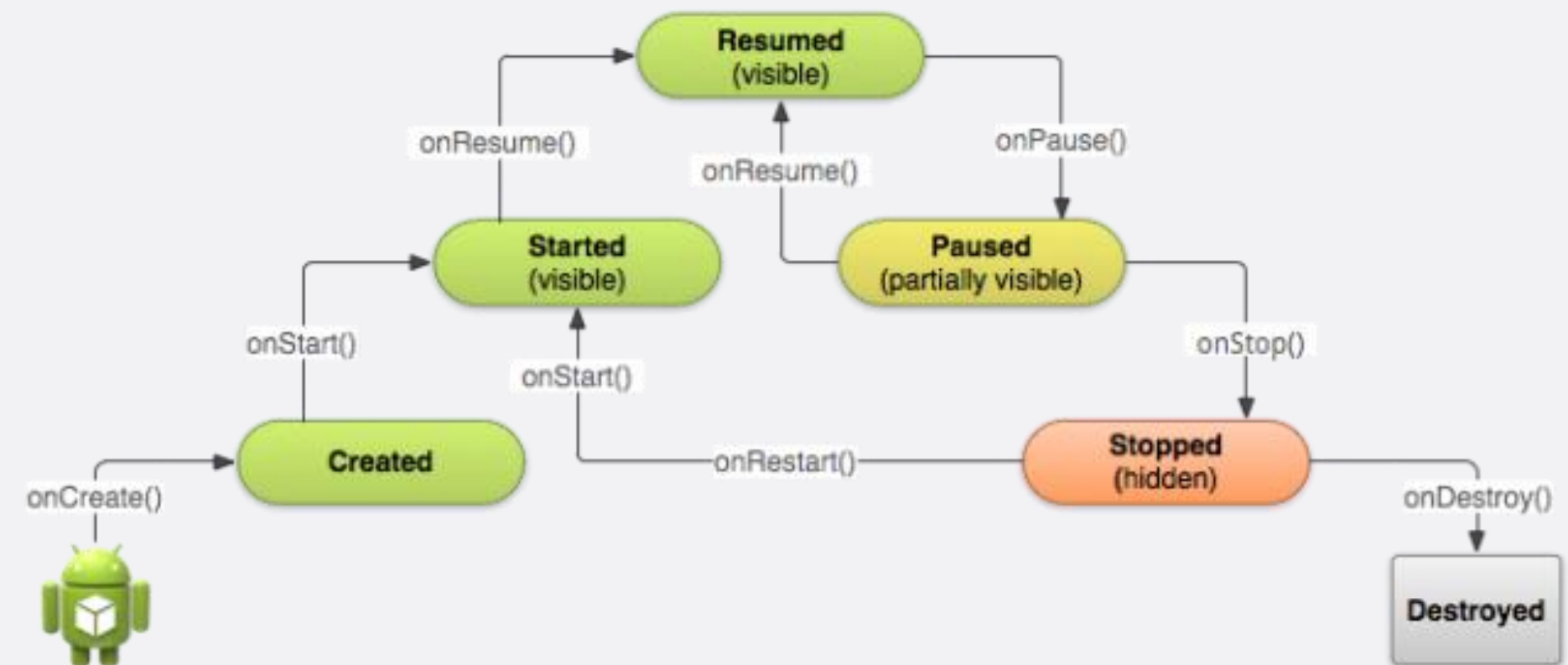
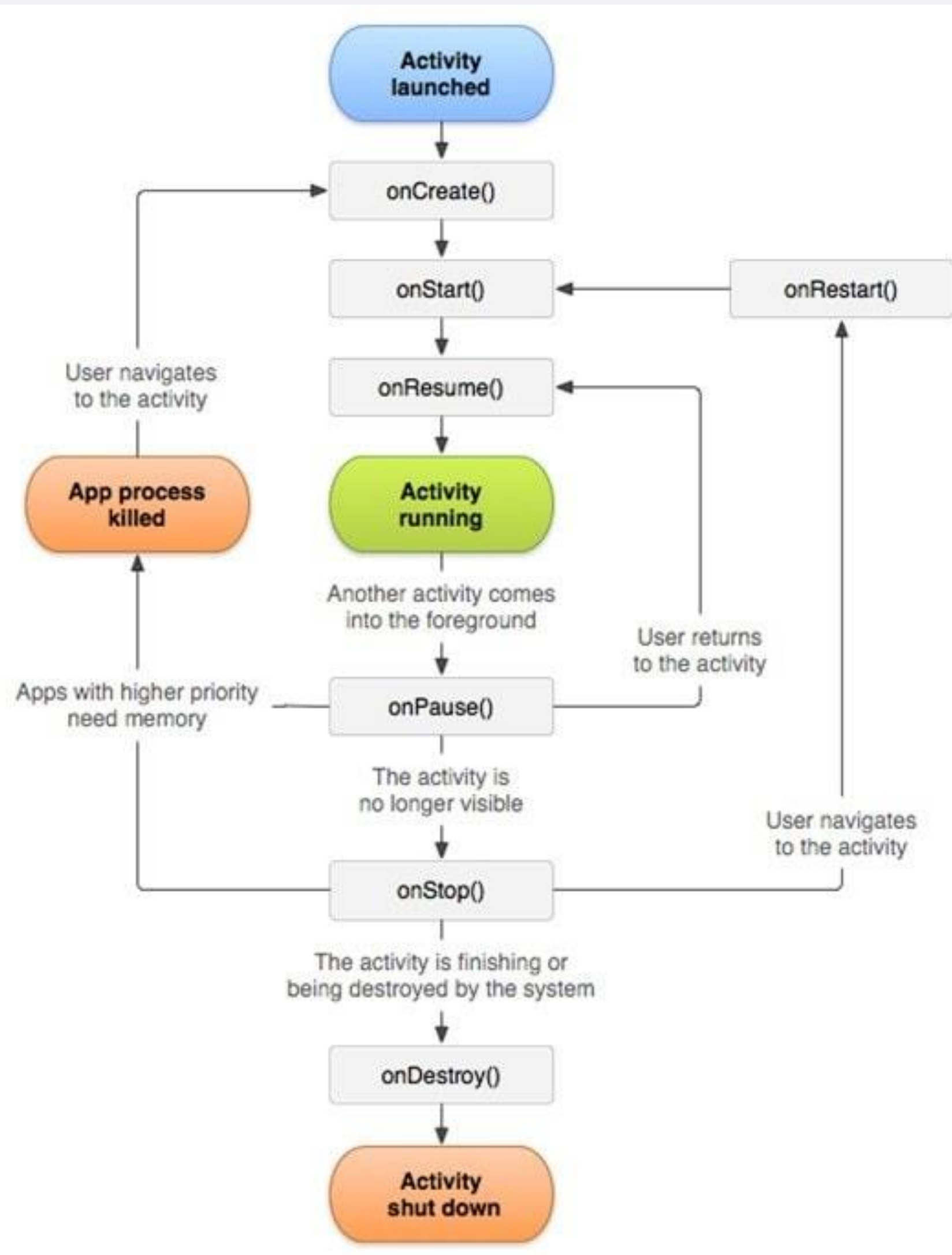
kotlin

```
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    HelloWorldTheme {
        Greeting("Android")
    }
}
```

kotlin

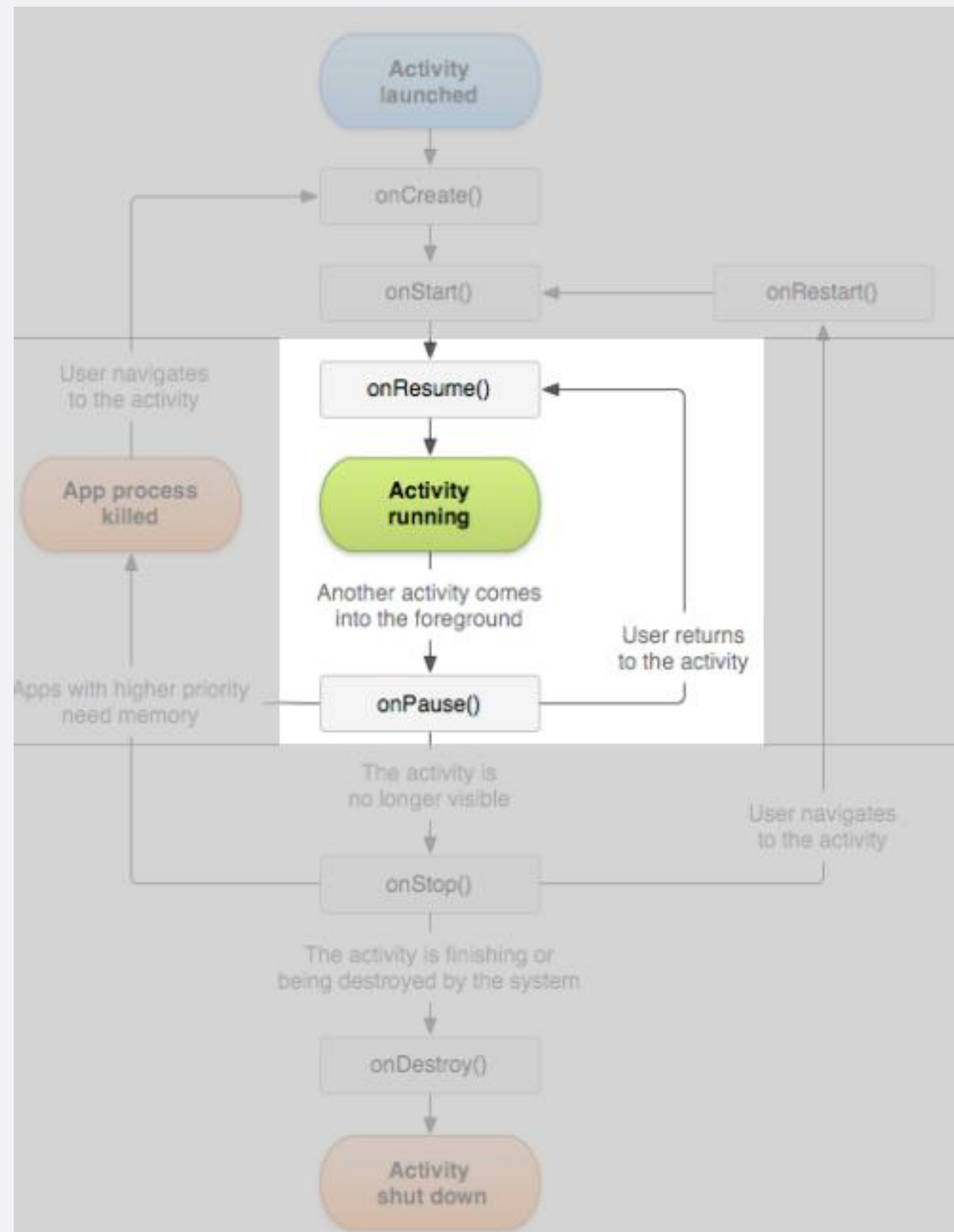
```
setContent {
    HelloWorldTheme {
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
            Greeting(
                name = "Android",
                modifier = Modifier.padding(innerPadding)
            )
        }
    }
}
```


Activity: ciclo di vita



Activity: ciclo di vita

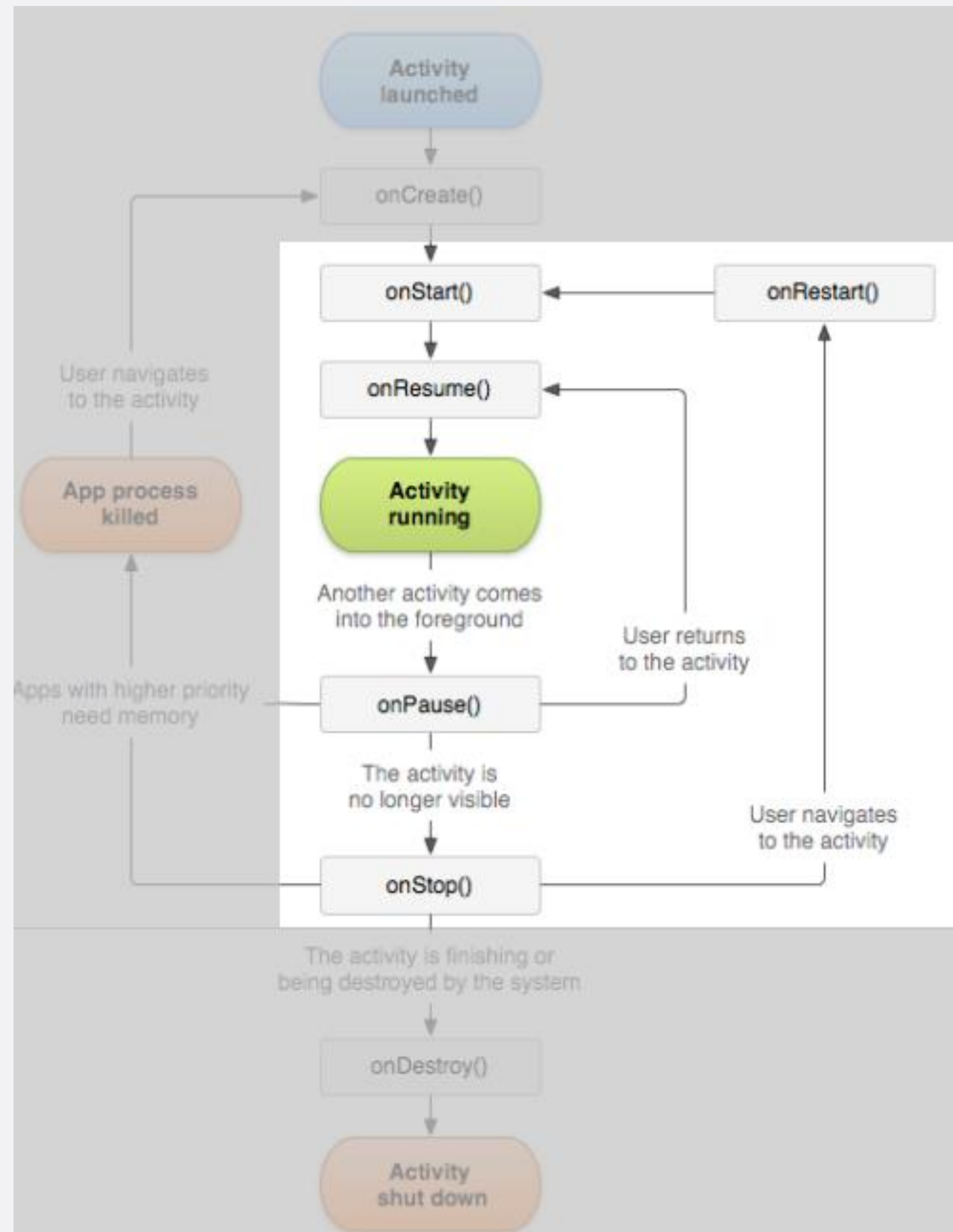
FOREGROUND



Activity: ciclo di vita

VISIBLE

- Quando interrotta, l'Activity dovrebbe rilasciare le risorse considerate «costose», come la rete o connessioni al database.
- Quando l'Activity riprende, si possono riacquistare tali risorse ricaricando lo stato precedente dell'Activity.



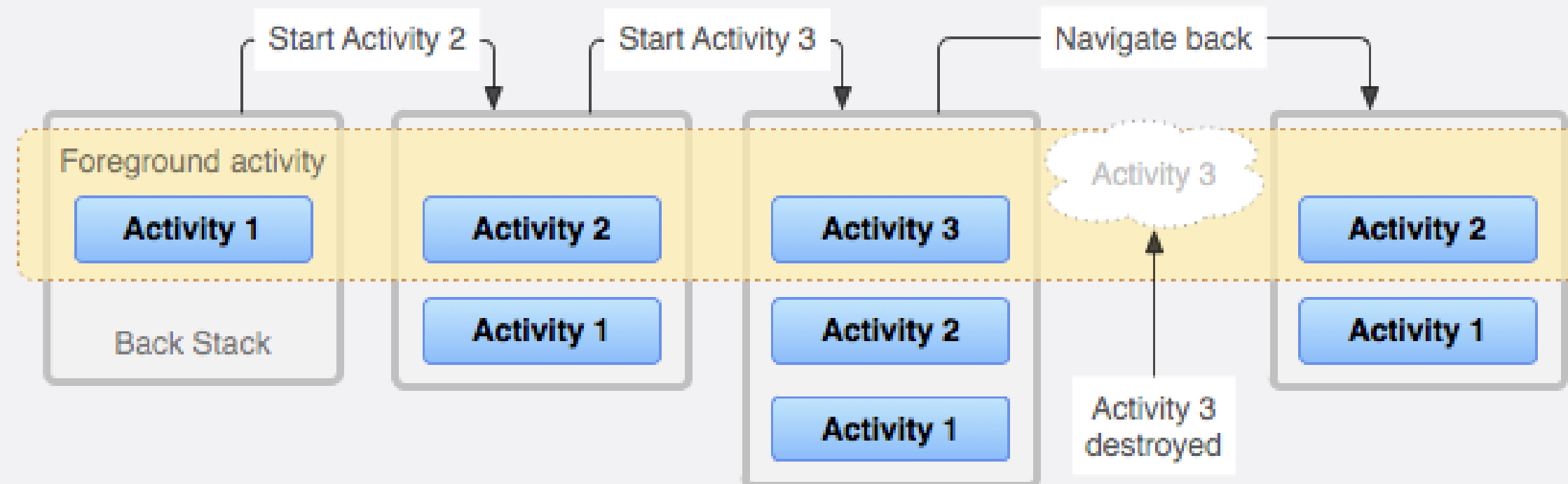
la classe Activity

- **onCreate(Bundle savedInstanceState)** - Eseguito al primo avvio dell'Activity. L'Activity viene creata. Qui vengono assegnate le configurazioni di base e definito quale sara' il layout dell'interfaccia.
- **onStart()** – Eseguito quando l'Activity viene avviata. L'Activity diventa visibile. E' il momento in cui si possono attivare funzionalita' e servizi che devono offrire informazioni all'utente.
- **onResume()** - Eseguito quando l'Activity ritorna in primo piano (foreground). L'Activity diventa la destinataria di tutti gli input dell'utente.

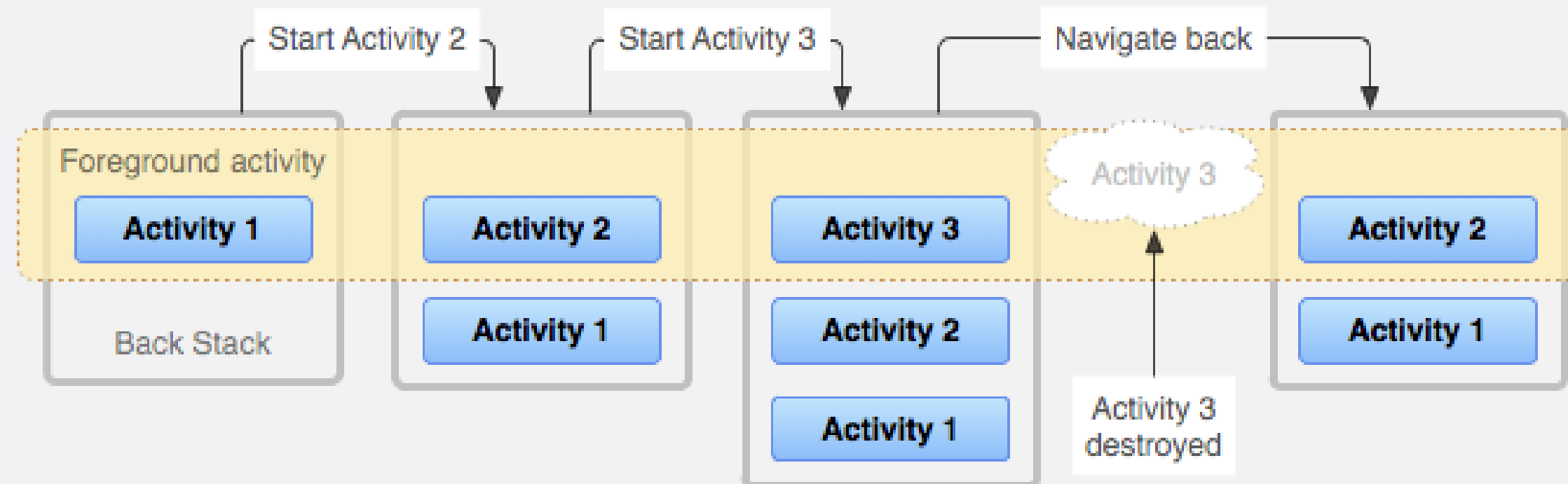
la classe Activity

- **onPause()** - Eseguito quando l'Activity smette di essere in primo piano (foreground) ed e' messa in secondo piano da un'altra Activity. Notifica la cessata interazione dell'utente con l'Activity.
- **onStop()** – Eseguito quando l'Activity viene sostituita da un'altra (ma e' ancora istanziata in memoria). Segna la fine della visibilita' dell'Activity.
- **onDestroy()** – Eseguito alla chiusura e deallocazione dell'Activity. Segna la distruzione dell'Activity.

Activity stack

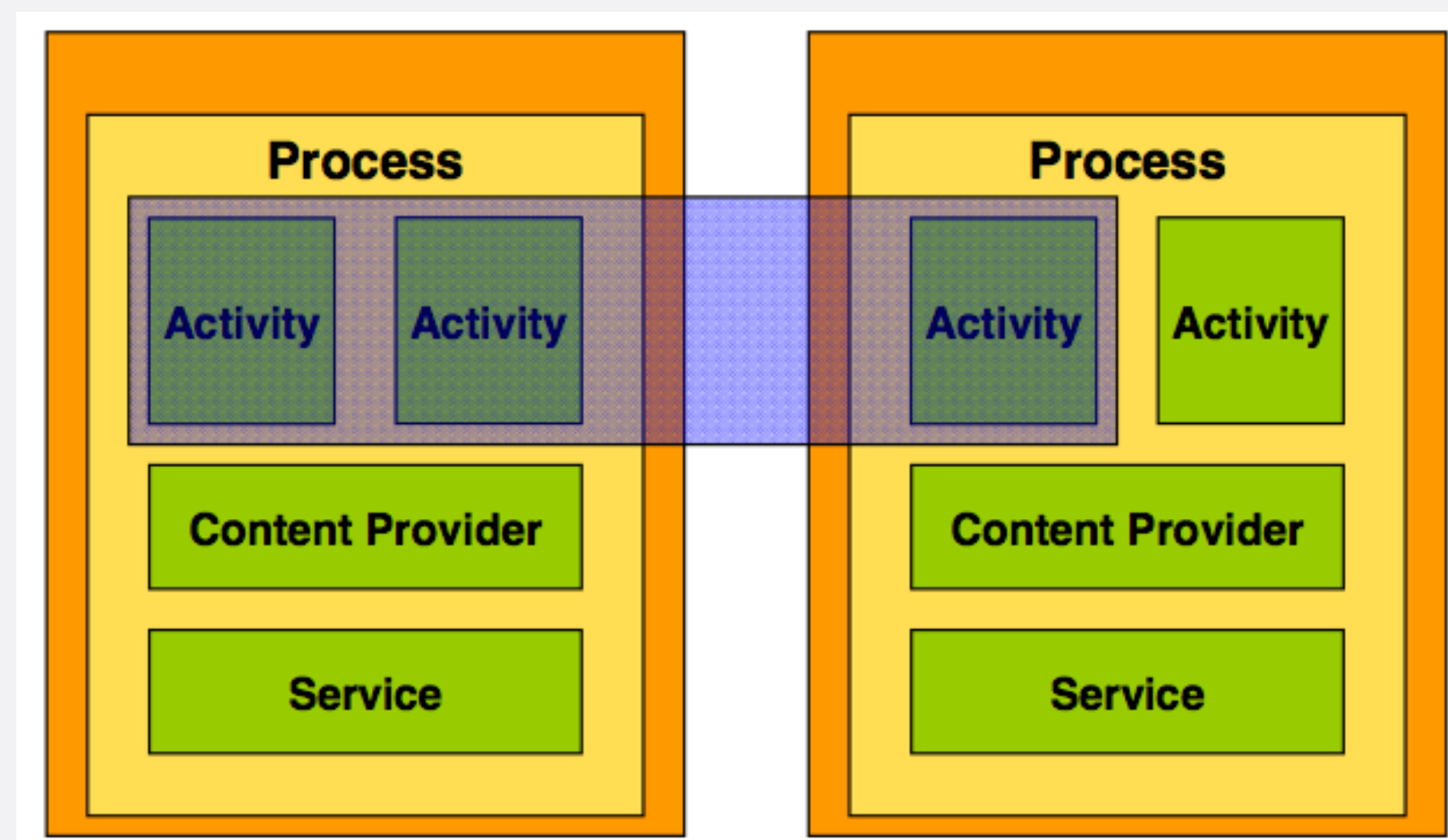


Activity stack



TASK

- Cio' che gli utenti vedono sotto forma di applicazione.
- Rappresenta una collezione di Activity della stessa applicazione.
- Hanno un loro stack.

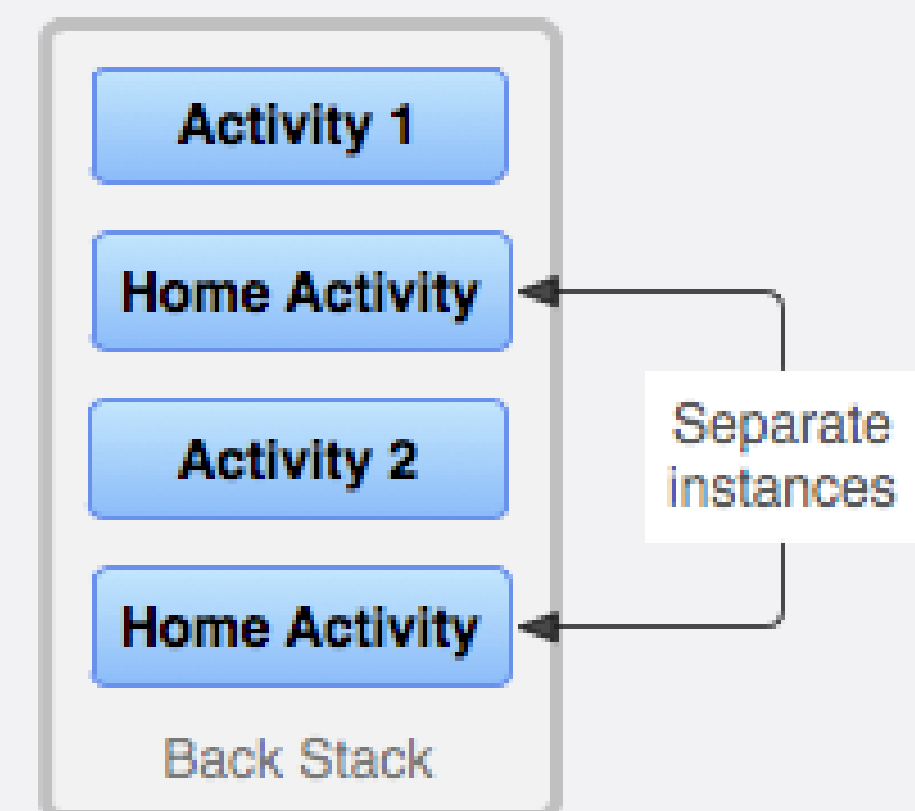


Task

- La Home e' il punto di partenza della maggior parte dei Task.
- Quando l'utente tocca una delle icone nel launcher (o nella schermata di Home), il task dell'applicazione associata viene posto in foreground.
- Se non esiste alcuna attivita' per l'applicazione selezionata (il che significa che l'applicazione non e' stata utilizzata di recente), viene creata una nuova «main» Activity nello stack di quell'applicazione.
- Se, invece, l'applicazione e' stata usata di recente, il suo task viene richiamato dalla memoria (ed in genere il suo stato viene ripristinato).

Task

- Un'applicazione definisce almeno un task... ma puo' definirne di piu'.
- Le Activity presenti all'interno di un task possono provenire da diverse applicazioni.
- I Task possono anch'essi essere spostati in background.
- **PUNTO DI ATTENZIONE: una stessa ACTIVITY puo' essere istanziata piu' volte.**



istanze multiple – gestione tramite il manifesto

```
<activity android:launchMode =  
    ["standard" | "singleTop" | "singleTask" | "singleInstance"] ..  
>
```

- **standard:** la stessa Activity viene creata piu' volte nello stack.

Esempio:

1) stack: A -> B -> C -> D

2) chiamo l'Activity B

3) stack: A -> B -> C -> D -> B

istanze multiple – gestione tramite il manifesto

```
<activity android:launchMode =  
    ["standard" | "singleTop" | "singleTask" | "singleInstance"] ..  
>
```

- **singleTop**: se un'Activity e' gia' presente come top-Activity nello stack, non viene creata una nuova istanza.

Esempio 1:

1) stack: A -> B -> C

2) chiamo l'Activity D

3) stack: A -> B -> C -> D

istanze multiple – gestione tramite il manifesto

```
<activity android:launchMode =  
    ["standard" | "singleTop" | "singleTask" | "singleInstance"] ..  
>
```

- **singleTop**: se un'Activity e' gia' presente come top-Activity nello stack, non viene creata una nuova istanza.

Esempio 2:

1) stack: A -> B -> C -> D

2) chiamo l'Activity D

3) stack: A -> B -> C -> D

istanze multiple – gestione tramite il manifesto

```
<activity android:launchMode =  
    ["standard" | "singleTop" | "singleTask" | "singleInstance"] ..  
>
```

- **singleTask**: una nuova istanza dell'Activity viene creata se e solo se questa Activity non esista già all'interno di un altro Task della vostra applicazione.

Esempio 1:

1) stack: A -> B -> C

2) chiamo l'Activity D

3) stack: A -> B -> C -> D

istanze multiple – gestione tramite il manifesto

```
<activity android:launchMode =  
    ["standard" | "singleTop" | "singleTask" | "singleInstance"] ..  
>
```

- **singleTask**: una nuova istanza dell'Activity viene creata se e solo se questa Activity non esista già all'interno di un altro Task della vostra applicazione.

Esempio 1:

1) stack: A -> B -> C -> D

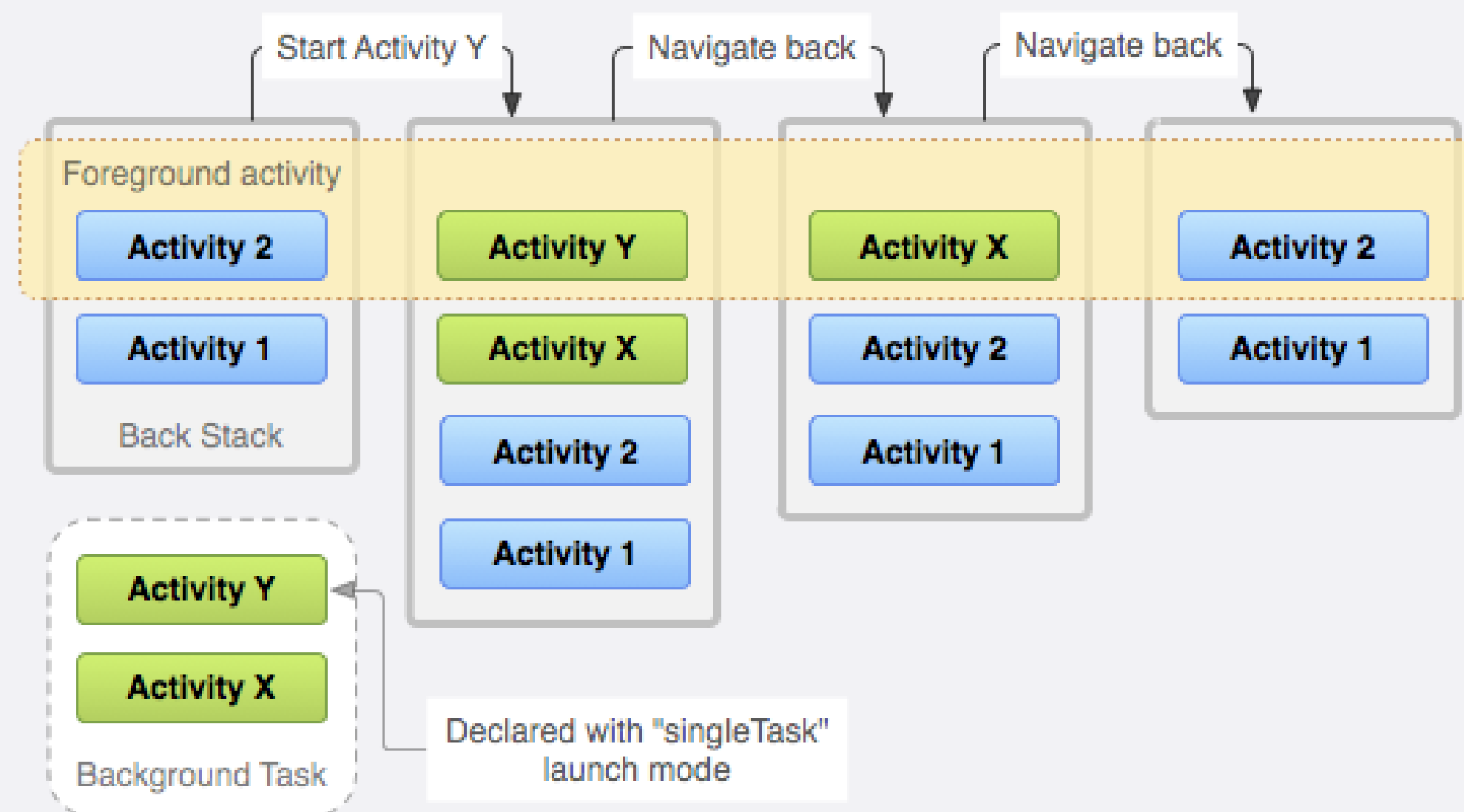
2) chiamo l'Activity B

3) stack: A -> B

istanze multiple – gestione tramite il manifesto

```
<activity android:launchMode =  
    ["standard" | "singleTop" | "singleTask" | "singleInstance"] ..  
>
```

- **singleTask**: una nuova istanza dell'Activity viene creata se e solo se questa Activity non esista già all'interno di un altro Task della vostra applicazione.



istanze multiple – gestione tramite il manifesto

```
<activity android:launchMode =  
    ["standard" | "singleTop" | "singleTask" | "singleInstance"] ..  
>
```

- **singleInstance**: come il **singleTask** con la differenza che l'Activity marcata come tale rimane la sola ad essere presente nel suo Task.

Esempio 1:

1) stack: A -> B -> C

2) chiamo l'Activity D

3) stack task 1: A -> B -> C stack task 2: D

4) chiamo l'Activity E

5) stack task 1: A -> B -> C -> E stack task 2: D

istanze multiple – gestione tramite il manifesto

```
<activity android:launchMode =  
    ["standard" | "singleTop" | "singleTask" | "singleInstance"] ..  
>
```

- **singleInstance**: come il **singleTask** con la differenza che l'Activity marcata come tale rimane la sola ad essere presente nel suo Task.

Esempio 2:

1) stack task 1: A -> B -> C stack task 2: D

2) chiamo l'Activity D

3) stack task 1: A -> B -> C stack task 2: D

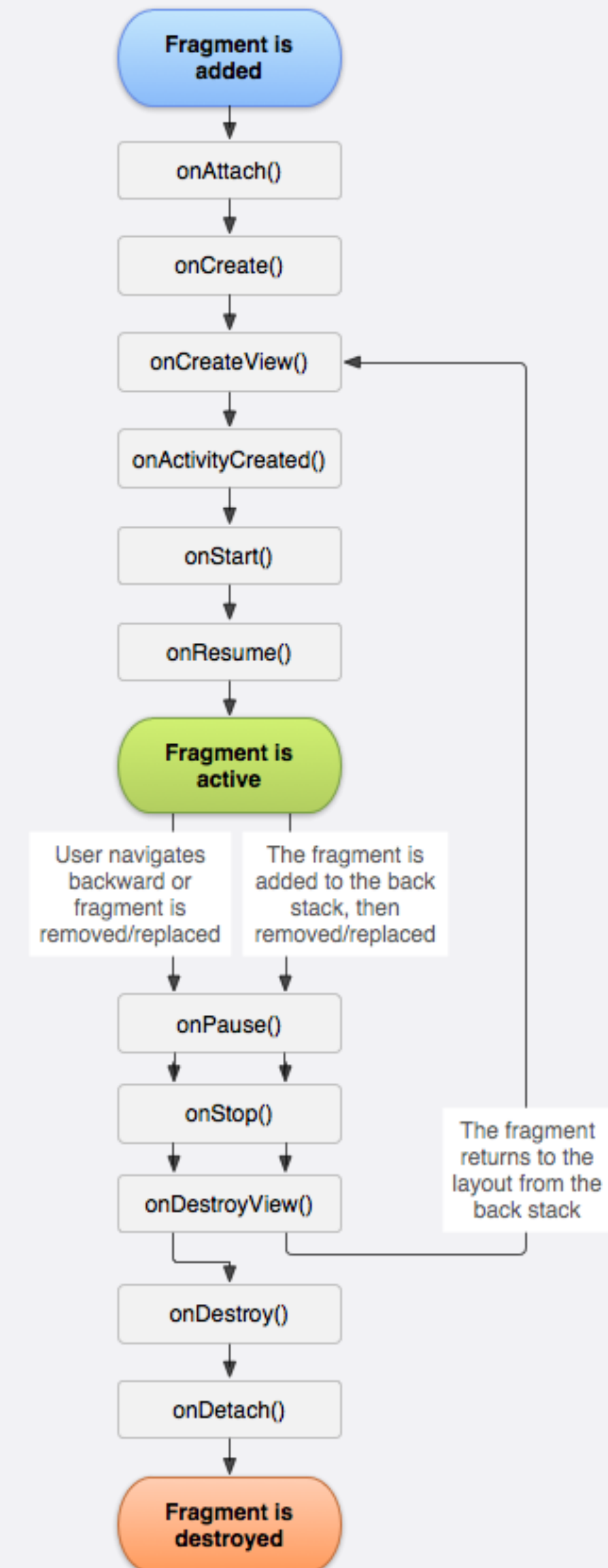
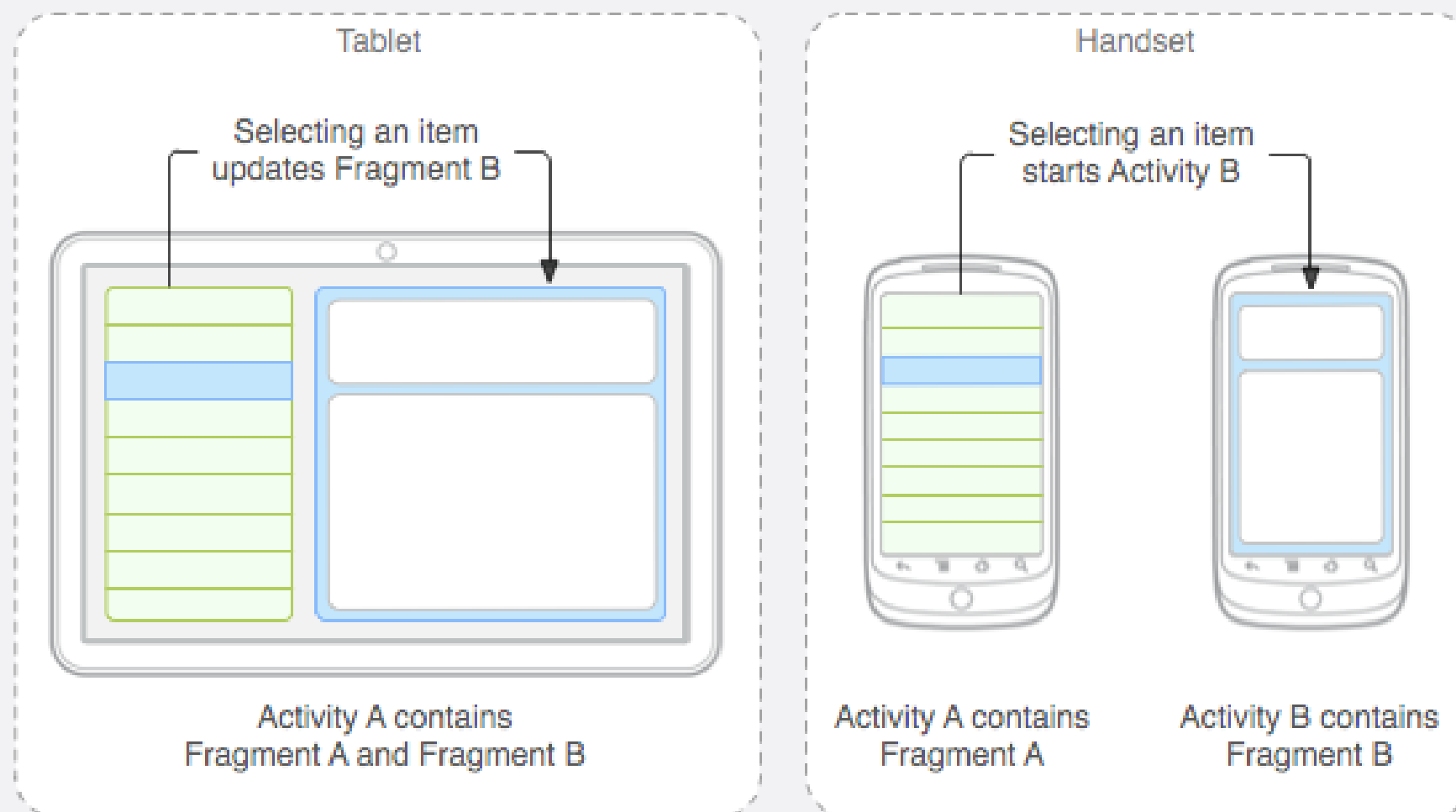
maggiori informazioni

- <https://developer.android.com/guide/components/activities/intro-activities>
- <https://developer.android.com/guide/components/activities/activity-lifecycle>
- <https://developer.android.com/guide/components/activities/state-changes>
- <https://developer.android.com/guide/components/activities/tasks-and-back-stack>

cenno ai Fragment

- Android non e' un ambiente a finestre
 - Si tratta di una scelta progettuale motivata dalla necessita' di poter essere installato con processori poco performanti, scarsa memoria e schermi video molto piccoli.
- A partire dalle versione 3.0, la diffusione dei tablet ha portato i progettisti a proporre una soluzione parziale al problema: i Fragment
- I Fragment somigliano molto ai Frame che potevano comporre una pagina Web (frameset) e rappresentano un comportamento o una parte dell'utente interfaccia in un'attivita'.
- E' possibile combinare piu' frammenti in un unica attivita' per creare un'interfaccia utente multi-riquadro e riutilizzare un frammento in piu' attivita'.
- Si puo' pensare a un frammento come una sezione modulare di un'Activity, che ha il suo ciclo di vita, riceve il suo eventi di input e che e' possibile aggiungere o rimuovere mentre l'Activity e' in esecuzione (un po 'come una «sub-Activity» che si puo' riutilizzare in diverse attivita').

cenno ai Fragment... ciclo di vita



Service

- Un Service svolge un ruolo "opposto" all'Activity (sebbene, a livello di ereditarietà di classe siano imparentati).
- Rappresenta un lavoro, generalmente lungo e continuato, che viene svolto interamente in background senza bisogno di interazione diretta con l'utente.
 - Ad esempio, un'applicazione che permette di avviare un audio che non si interrompe alla chiusura della sua interfaccia con tutta probabilità basa il suo funzionamento su un Service.
- I loro usi sono dei più disparati e, a seconda dei casi, l'utente potrebbe anche non notare il loro avvio ottenendone però i benefici.
- Da un punto di vista strutturale, i Service sono di due tipologie: **started** e **bounded**.
 - un Service è **started** quando un'applicazione ha bisogno di svolgere attività in background, mirate ad uno scopo specifico, fino al loro completamento.
 - un Service è **bounded** quando viene attivato solo nel caso in cui un'altra applicazione abbia bisogno di connettersi a loro. Si tratta di un tipo di Service che permette l'interazione tra processi differenti e risponde ad una logica simile a quella delle API nei servizi web.
- Si noti inoltre che anche questo settore è stato rivoluzionato negli anni, in particolare con la nascita dei JobScheduler che permettono lavori in background con un uso parsimonioso della batteria.

Content Provider

- Un Content Provider nasce con lo scopo della condivisione di dati tra applicazioni.
- La sua finalita' richiama quel principio di sicurezza dell'applicazione di cui parleremo piu' avanti.
- Questi componenti permettono di condividere, nell'ambito del sistema, contenuti custoditi in un database, su file o reperibili mediante accessi in Rete.
- Tali contenuti potranno essere usati da altre applicazioni senza invadere lo spazio di memoria ma stabilendo quel dialogo "sano" del quale parleremo.

Broadcast Receiver

- Un Broadcast Receiver e' un componente che reagisce ad un invio di messaggi a livello di sistema, appunto in broadcast, con cui Android notifica l'avvenimento di un determinato evento, ad esempio l'arrivo di un SMS o di una chiamata o sollecita l'esecuzione di azioni.
- Questi componenti come si puo' immaginare sono particolarmente utili per la gestione istantanea di determinate circostanze speciali.
- I Broadcast Receiver non utilizzano interfaccia grafica sebbene possano inoltrare notifiche alla barra di stato per avvisare l'utente dell'avvenimento.
- Inoltre la loro esecuzione dovrebbe essere istantanea delegando a Service o JobScheduler eventuali operazioni da attivare.

Intent

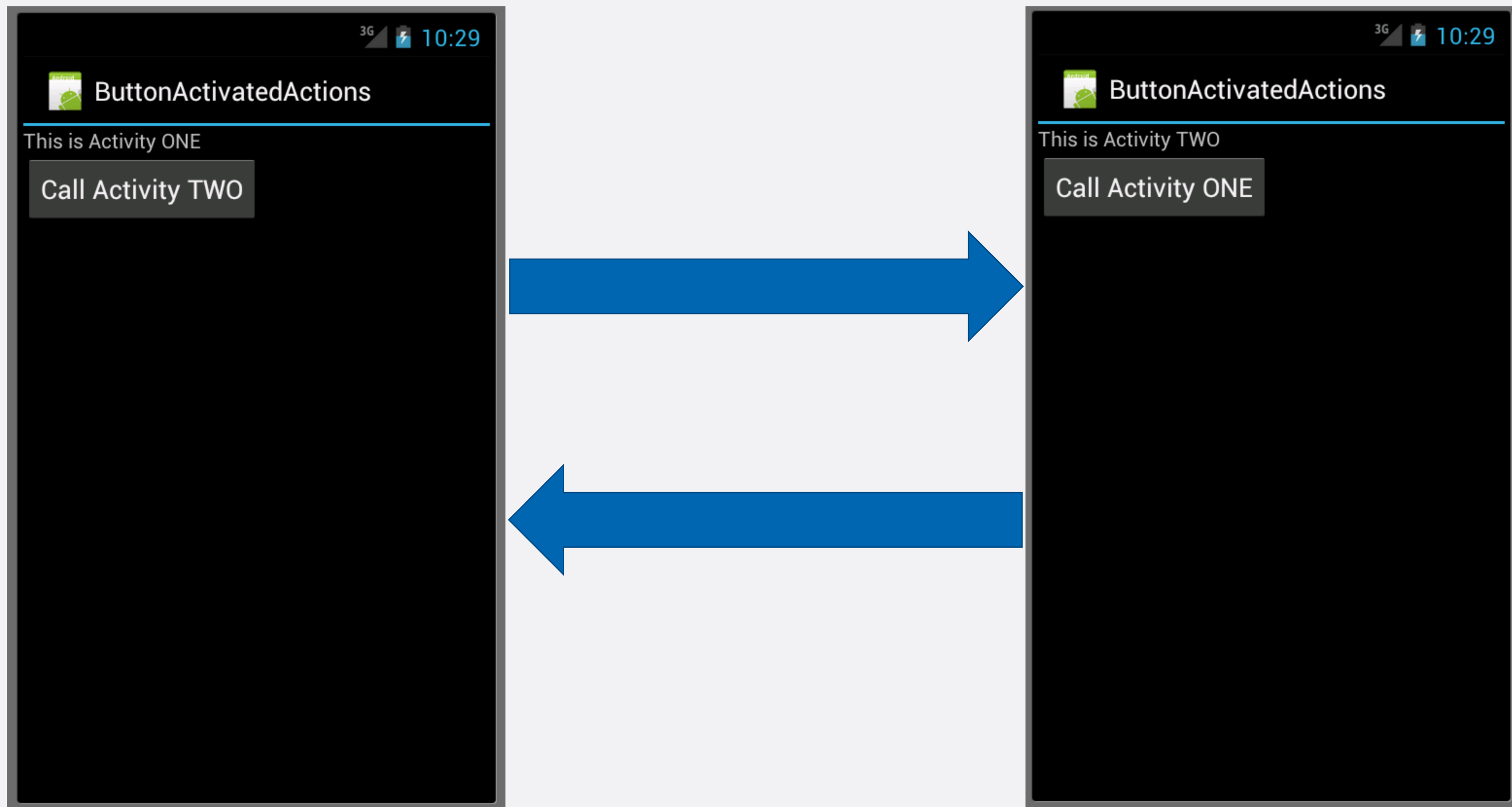
- Una Activity Android non può indiscriminatamente chiamare altre Activity
 - Questo meccanismo esiste sia per ragioni di sicurezza, sia per favorire il riuso di componenti, che viene mediato dal sistema.
- Per Intent si definisce un oggetto corrispondente ad un messaggio col quale si richiede l'attivazione di una Activity (o anche un servizio o un receiver)
- Intent sottintende un potentissimo strumento di comunicazione di Android.
 - avviare un'Activity;
 - avviare un Service;
 - inviare un messaggio in broadcast che può essere ricevuto da ogni applicazione.

Intent

`startActivity (Intent x)` (metodo della classe Activity)

- avvia una nuova Activity, che verra' posizionata nella parte superiore dello stack.
- accetta un singolo argomento che descrive l'Activity da eseguire.
- un intento e' una descrizione astratta di un'operazione da eseguire.

Intent



Intent

- Il modo piu' semplice per avviare da programma un'altra Activity e' con explicit intent:

```
val intent = Intent(this, NextActivity::class.java)  
startActivity(intent)
```
- Se, invece, vogliamo far partire un'Activity che svolga un particolare compito senza conoscere staticamente la classe che la implementa (implicit intent):

```
val intent = Intent(Intent.ACTION_SEND)  
startActivity(intent)
```
- In questo caso si chiede di avviare una Activity che abbia settato il filtro `ACTION_SEND`
- Per passare dati da una Activity ad un'altra si può utilizzare il metodo `putExtra` di `Intent` (passaggio per valore)

Intent espliciti

```
val intent = Intent(Context c1, Class c2);
```

- Da ricordare che un contesto e' un wrapper per le informazioni globali relative all'ambiente applicativo e, come sappiamo Activity e' una sottoclasse di Context

Intent – cosa accade nel ciclo di vita di una Activity?

- Il passaggio da un'Activity ad un'altra coinvolge i cicli di vita di entrambe.
- La prima viene messa a riposo, passa almeno per **onPause** (cessazione interazione con l'utente) e **onStop** (activity non piu' visibile).
- La seconda percorre la catena di creazione **onCreate–onStart–onResume**.
- Ma in che ordine avviene tutto cio'? La priorit  del sistema   il mantenimento della fluidit  della user-experience. Per questo la consecutio delle operazioni sara':
 - 1) la prima Activity passa per **onPause** e viene fermata in stato Paused;
 - 2) la seconda Activity va in Running venendo attivata completamente. In tale maniera l'utente potra' usarla al piu' presto non subendo tempi di ritardo;
 - 3) mentre l'utente sta gia' usando la seconda Activity, il sistema puo' invocare **onStop** sulla prima.

Intent – gestione stack

- E' possibile gestire lo stack del task direttamente durante la chiamata degli intenti:
- `intent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);`
nessun lunch mode equivalente. Se l'Activity gia' esiste nello stack dell'Activity chiamante, tutte quelle davanti ad essa vengono distrutte. A differenza della modalita' «singleTask», viene controllato **solo il task corrente e non tutti i task attivi**.
- `intent.setFlags(Intent.FLAG_ACTIVITY_SINGLE_TOP);`
come «singleTop» lunch mode.
- `intent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TASK);`
nessun lunch mode equivalente. Svuota lo stack del task ed istanzia la nuova Activity.
- `intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK);`
come «singleTask» lunch mode.

Intent – passaggio dati: gli Extra



- **MainActivity** contiene un semplice form di login. Dopo aver inserito username e password viene controllata la validità dei dati ed in caso positivo viene invocata l'apertura di un'altra Activity;
- **SecretActivity** è l'area accessibile solo mediante login e contiene, si suppone, dati riservati.

Intent – passaggio dati: gli Extra



■ MainActivity

```
val intent = Intent(this, SecretActivity::class.java)
intent.putExtra("USERNAME", "Topolino")
intent.putExtra("PASSWORD", "Paperino")
startActivity(intent)
```

■ SecretActivity

```
val name = intent.getStringExtra("USERNAME")
val password = intent.getStringExtra("PASSWORD")
```


Intent – best practice

```
val intent = Intent(this, SecondActivity::class.java).apply {  
    putExtra("USER_NAME", "Marco")  
    putExtra("USER_AGE", 30)  
}  
startActivity(intent)
```

Intent – gestione del risultato

```
// nella activity chiamante
```

```
val launcher =  
registerForActivityResult(ActivityResultContracts.StartActivityForResult()) {  
    result ->  
        if (result.resultCode == RESULT_OK) {  
            val data = result.data?.getStringExtra("RESULT_KEY")  
            // uso dei dati restituiti  
        }  
}
```

```
// richiamo della seconda activity quando necessario  
val intent = Intent(this, SecondActivity::class.java)  
launcher.launch(intent)
```

Intent – gestione del risultato

// nella seconda activity quando e' pronta per restituire qualcosa

```
val resultIntent = Intent().apply {  
    putExtra("RESULT_KEY", "some value")  
}  
setResult(RESULT_OK, resultIntent)  
finish()
```